📁 Arduino – Sistema Purificatore OffGas

Documento operativo e descrittivo

Questo documento descrive l'implementazione del firmware Arduino utilizzato nel sistema **OffGas**.

Il firmware ha il compito di gestire:

1️⃣ lettura del sensore di gas MQ-2
2️⃣ comunicazione Bluetooth con il Bridge
3️⃣ attivazione della ventola
4️⃣ gestione dello stato del dispositivo.

Arduino rappresenta il **livello hardware (edge device)** del sistema IoT OffGas.

Panoramica del ruolo di Arduino

Nel sistema OffGas Arduino è collegato a:

- sensore **MQ-2**
- modulo **Bluetooth HC-05**
- **ventola di ventilazione**
- **LED di stato**

Arduino **non prende decisioni autonome** sulla qualità dell'aria.

La logica di controllo è distribuita:

- Arduino → acquisizione sensore
- Bridge Python → gateway hardware
- Node-RED → logica decisionale.

Arduino si limita quindi a:

- inviare il valore del gas
- ricevere i comandi di accensione o spegnimento della ventola.

Architettura hardware

I componenti principali collegati ad Arduino sono:

Componente	Funzione
MQ-2	sensore gas
HC-05	comunicazione Bluetooth
Ventola	purificazione aria
LED	indicatore stato ventola

Definizione dei PIN

Nel firmware i pin utilizzati sono definiti all'inizio del programma.

```
const int mq2Pin = A1;
const int bluetoothRxPin = 10;
const int bluetoothTxPin = 11;
const int PIN_VENTOLA = 9;
```

Significato:

Pin	Funzione
A1	ingresso analogico sensore MQ-2
10	RX Bluetooth
11	TX Bluetooth
9	controllo ventola

Comunicazione Bluetooth

La comunicazione con il Bridge avviene tramite **SoftwareSerial**.

```
#include <SoftwareSerial.h>
```

Il modulo Bluetooth HC-05 viene inizializzato con:

```
SoftwareSerial bluetooth(bluetoothRxPin, bluetoothTxPin);
```

Velocità di comunicazione:

```
bluetooth.begin(9600);
```

Questa velocità deve essere **uguale a quella utilizzata dal Bridge Python**.

Nel Bridge infatti la seriale viene aperta con:

```
serial.Serial(
    port=self.port,
    baudrate=self.baud_rate
)
```

dove:

```
BAUD_RATE = 9600
```

Parametri operativi

Il firmware utilizza una lettura periodica del sensore.

```
const unsigned long intervalloLettura = 5000;
```

Questo valore rappresenta l'intervallo tra due letture successive:

5000 ms = 5 secondi

Quindi Arduino invia un nuovo valore del gas **ogni 5 secondi**.

Variabili di stato

Il firmware mantiene alcune variabili per tracciare lo stato del sistema.

```
unsigned long ultimoTempoLettura = 0;
bool fanState = false;
```

Significato:

Variabile	Funzione
ultimoTempoLettura	ultimo momento di lettura sensore
fanState	stato corrente della ventola

Inizializzazione del sistema (setup)

La funzione setup() inizializza i componenti hardware.

```
void setup() {
  pinMode(mq2Pin, INPUT);
  pinMode(PIN_VENTOLA, OUTPUT);
  pinMode(LED_BUILTIN, OUTPUT);
}
```

Vengono inoltre inizializzate le comunicazioni seriali.

```
Serial.begin(9600);
bluetooth.begin(9600);
```

La seriale USB è utilizzata per il **debug**.

Messaggio di avvio:

```
Serial.println("Arduino OffGas Ready.");
```

Ciclo principale del firmware

Arduino utilizza il classico ciclo infinito:

```
void loop()
```

All'interno del ciclo vengono gestite due funzioni principali:

1. invio del valore del sensore
2. ricezione dei comandi dal Bridge.

---

1 Invio periodico del valore MQ-2

Il valore del sensore viene letto ogni 5 secondi.

La temporizzazione è gestita tramite la funzione millis().

```
unsigned long now = millis();
```

Controllo dell'intervallo:

```
if (now - ultimoTempoLettura >= intervalloLettura)
```

Se il tempo è trascorso viene effettuata una nuova lettura.

Lettura del sensore

Il sensore MQ-2 produce un segnale analogico.

La lettura avviene tramite:

```
int valore = analogRead(mq2Pin);
```

Il valore restituito è compreso tra:

0 → 1023

Invio del valore al Bridge

Il valore del gas viene inviato tramite Bluetooth con il formato:

MQ2:<valore>

Nel codice:

```
bluetooth.print("MQ2:");
bluetooth.println(valore);
```

Esempio messaggio:

MQ2:178

Questo formato è stato scelto per semplificare il parsing nel Bridge.

Nel Bridge infatti la funzione di lettura è:

```
read_val = self.ser.readline().decode('utf-8').strip()

if read_val.startswith("MQ2:"):
```

Il Bridge estrae il valore con:

```
val_gas = int(read_val.split(':',1)[1])
```

Debug sulla seriale

Il valore del gas viene stampato anche sulla seriale USB.

```
Serial.print("MQ2:");
Serial.println(valore);
```

Questo permette di monitorare il sensore tramite **Serial Monitor di Arduino IDE**.

2️⃣ Ricezione comandi dal Bridge

Arduino può ricevere comandi tramite Bluetooth.

Controllo presenza dati:

```
if (bluetooth.available())
```

Lettura del comando:

```
String comando = bluetooth.readStringUntil('\n');
comando.trim();
```

Il Bridge invia i comandi terminati da newline:

```
self.bluetooth.send_command("FAN_ON\n")
```

Comando accensione ventola

Se il comando ricevuto è:

```
FAN_ON
```

Arduino accende la ventola.

Nel codice:

```
if (comando.equalsIgnoreCase("FAN_ON")) {
```

Attivazione ventola:

```
digitalWrite(PIN_VENTOLA, HIGH);
```

Attivazione LED:

```
digitalWrite(LED_BUILTIN, HIGH);
```

Aggiornamento stato:

fanState = true;

Messaggio debug:

Serial.println("Ventola ON (da Bridge)");

Comando spegnimento ventola

Se il comando ricevuto è:

FAN_OFF

Arduino spegne la ventola.

Nel codice:

else if (comando.equalsIgnoreCase("FAN_OFF"))

Spegnimento ventola:

digitalWrite(PIN_VENTOLA, LOW);

Spegnimento LED:

digitalWrite(LED_BUILTIN, LOW);

Aggiornamento stato:

fanState = false;

Comunicazione con il Bridge

La comunicazione tra Arduino e Bridge segue questo schema.

Arduino → Bridge

Invio valore sensore:

MQ2:178

Bridge → Arduino

Comandi supportati:

FAN_ON
FAN_OFF

Il Bridge invia questi comandi quando riceve decisioni da Node-RED.

Nel codice Python del Bridge:

self.fan_controller.force_on()

che invia:

self.bluetooth.send_command("FAN_ON\n")

Flusso completo del sistema

Il funzionamento del sistema OffGas è il seguente.

1. Arduino legge il sensore MQ-2
2. Arduino invia il valore al Bridge
3. Bridge pubblica telemetria MQTT
4. Node-RED analizza i dati
5. Node-RED decide lo stato della ventola
6. Bridge invia comando ad Arduino
7. Arduino attiva o spegne la ventola.

Vantaggi dell'architettura

Separare le responsabilità tra Arduino, Bridge e Node-RED offre diversi vantaggi.

Firmware semplice

Arduino esegue solo operazioni hardware.

Logica centralizzata

La logica di controllo è gestita da Node-RED.

Scalabilità

Il sistema può essere esteso facilmente a più garage.

Facilità di manutenzione

La logica può essere modificata senza aggiornare il firmware Arduino.

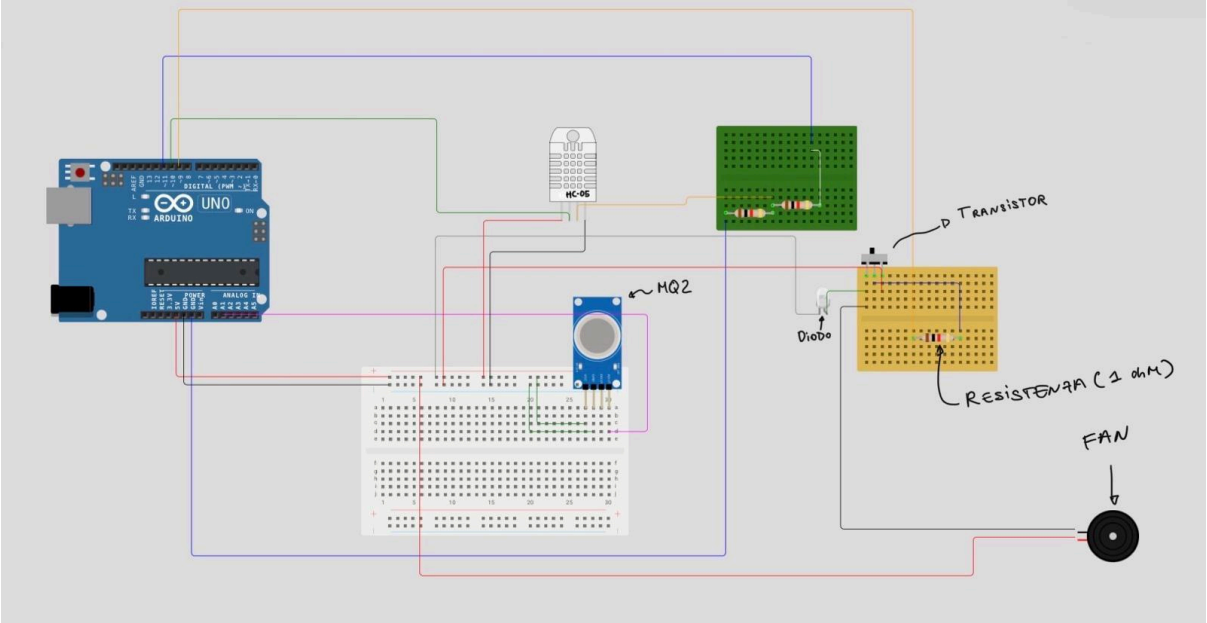
Checklist funzionale

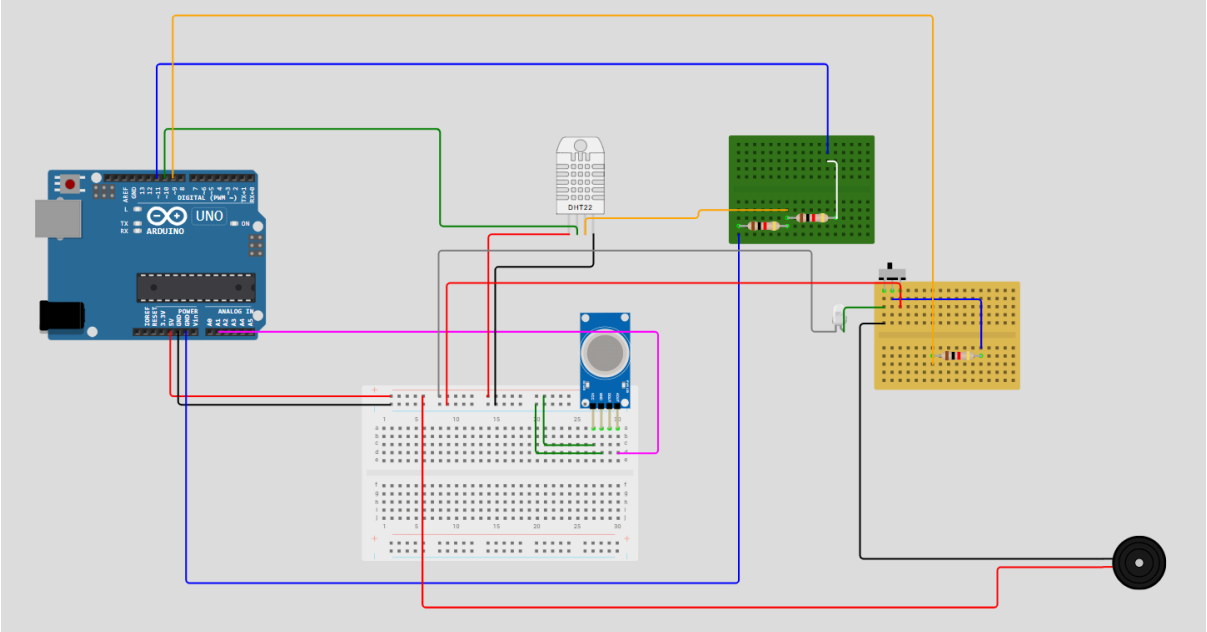
Il firmware Arduino implementa le seguenti funzionalità:

✔ Lettura del sensore MQ-2
✔ Invio valore gas via Bluetooth
✔ Ricezione comandi dal Bridge
✔ Controllo della ventola
✔ Indicazione visiva tramite LED
✔ Debug tramite seriale USB.

CIRCUITO

(https://wokwi.com/projects/458136524962164737)





Descrizione del circuito hardware – Sistema OffGas

Panoramica

Il prototipo hardware del sistema OffGas è basato su **Arduino Uno** ed è progettato per monitorare la concentrazione di gas in un garage e attivare un sistema di ventilazione quando necessario.

Il sistema è composto da:

- un sensore di gas **MQ-2**
- un **modulo Bluetooth HC-05** per la comunicazione con il Bridge Python
- una **ventola DC** per la ventilazione
- un **transistor NPN** per il controllo della ventola
- un **diodo flyback** per la protezione del circuito
- resistenze di pull-up e di limitazione della corrente
- breadboard e cablaggio

Arduino funge da **nodo edge IoT**, acquisendo i dati dei sensori e ricevendo comandi dal Bridge per controllare la ventola.

Componenti utilizzate

Componente	Funzione
Arduino Uno	microcontrollore principale
Sensore MQ-2	rilevamento gas combustibili
Modulo Bluetooth HC-05	comunicazione con il Bridge Python
Ventola DC	ventilazione del garage
Transistor NPN	pilotaggio della ventola
Diodo flyback	protezione contro tensioni inverse
Resistenze	pull-up e limitazione corrente
Breadboard	prototipazione

Collegamenti dei sensori

Sensore gas MQ-2

Il sensore MQ-2 è utilizzato per misurare la concentrazione di gas nell'aria.

Collegamenti:

Pin MQ-2	Collegamento
VCC	5V Arduino
GND	GND Arduino
AO	A1 Arduino

Il pin **AO** fornisce un valore analogico proporzionale alla concentrazione di gas.

Comunicazione Bluetooth

La comunicazione con il sistema software è realizzata tramite il **modulo HC-05**.

Il modulo utilizza una comunicazione **seriale UART** con Arduino.

Collegamenti principali:

HC-05	Arduino
VCC	5V
GND	GND
TX	pin seriale Arduino
RX	pin seriale Arduino

Il modulo consente al Bridge Python di:

- ricevere i valori del sensore MQ-2
- inviare comandi di controllo della ventola

Circuito di controllo della ventola

La ventola non può essere collegata direttamente ad Arduino perché richiede una corrente superiore a quella fornibile dai pin digitali.

Per questo viene utilizzato un **transistor NPN come interruttore elettronico**.

Collegamenti del transistor

Pin transistor	Collegamento
Base	Arduino tramite resistenza
Collector	terminale negativo della ventola
Emitter	GND

La **resistenza sulla base** limita la corrente proveniente dal pin digitale di Arduino.

Collegamento della ventola

Ventola	Collegamento
positivo	alimentazione 5V
negativo	collector del transistor

Quando Arduino attiva il pin di controllo, il transistor conduce e la ventola si accende.

Diodo flyback

Il circuito include un **diodo flyback** collegato in parallelo alla ventola.

Funzione:

- proteggere il transistor
- proteggere Arduino
- assorbire la tensione inversa generata dal motore quando si spegne

Orientamento:

- catodo verso il lato positivo
- anodo verso il lato del transistor

Alimentazione

L'alimentazione del circuito è fornita dall'Arduino attraverso il pin **5V**.

La breadboard distribuisce:

- linea **5V**
- linea **GND**

a tutti i componenti del circuito.

Funzionamento del sistema

Il funzionamento del prototipo è il seguente:

1. Il sensore **MQ-2** misura la concentrazione di gas.
2. Arduino legge il valore analogico dal pin **A1**.
3. Il valore viene inviato via **Bluetooth (HC-05)** al Bridge Python.
4. Il Bridge pubblica i dati su **MQTT** verso Node-RED.
5. Node-RED analizza i dati e decide se attivare la ventilazione.
6. Se necessario, il Bridge invia il comando **FAN_ON** ad Arduino.
7. Arduino attiva il transistor che alimenta la **ventola DC**.

In caso contrario viene inviato il comando **FAN_OFF**, spegnendo il sistema di ventilazione.

Ruolo del prototipo nel sistema OffGas

Il prototipo rappresenta il **garage locale (G1)** nel sistema OffGas.

Il dispositivo:

- misura la qualità dell'aria
- invia i dati alla piattaforma IoT
- riceve comandi di controllo
- attiva il sistema di ventilazione quando necessario

In questo modo il garage diventa un **nodo IoT intelligente** integrato nella rete di monitoraggio distribuita.